

Phase 4: Agents & Tools — The Complete Beginner's Guide

Stack used throughout this guide: Node.js + TypeScript + OpenAI (gpt-4o-mini)

How to Use This Guide

This is a standalone deep-dive into one phase of your LangChain roadmap: **Agents & Tools**. It assumes you've already been through Foundation (Models, Prompts, Output Parsers) and Core (Runnables, LCEL, Chains, Memory) — so you're comfortable calling a model and piping things together. What this guide does **not** assume is that you know anything about agents. We start from zero on that front.

Every topic below follows the same shape:

1. **Concept and Theory** — plain-language explanation, with analogies.
2. **Practical Lab** — step-by-step code you type and run yourself, with explanations of *why* each line exists.
3. **Knowledge Check** — summary, key takeaways, practice questions, and a small exercise.

By the end, you'll build a **Final Project**: a multi-tool assistant that can do math, search the web, and explain its own reasoning — all running on your own machine.

Chapter 0: One-Time Project Setup

Before Chapter 1, set up a single project folder. You'll reuse this for every lab in this guide — each lab is just a new file inside it.

Why we do this first

Every lab needs the same model, the same API key, and the same TypeScript config. Setting this up once avoids repeating boilerplate seven times and mirrors how a real project is structured: one `package.json`, one `.env`, many small scripts.

Step-by-step

Step 1 — Create the folder and initialize npm

```
mkdir agents-and-tools-lab
cd agents-and-tools-lab
npm init -y
```

`npm init -y` generates a `package.json` with default values, skipping the interactive prompts. This file is what npm uses to track which packages your project depends on.

Step 2 — Install the packages we'll need across this whole guide

```
npm install langchain @langchain/core @langchain/openai @langchain/tavily zod
```

```
dotenv
npm install -D typescript ts-node @types/node
```

What each package is for:

Package	Why we need it
langchain	Contains the agent-building helpers: <code>createReactAgent</code> , <code>createToolCallingAgent</code> , <code>AgentExecutor</code> , and <code>pull</code> (for fetching prompts from LangChain Hub).
@langchain/core	The low-level building blocks: <code>tool()</code> , prompt templates, message types.
@langchain/openai	The <code>ChatOpenAI</code> class — our connection to OpenAI's models.
@langchain/tavily	A ready-made web search tool we'll give our agent in Chapter 5.
zod	A schema library. We use it to describe exactly what input each of our tools expects, so the agent (and TypeScript) know what's valid.
dotenv	Loads secrets (API keys) from a <code>.env</code> file instead of hardcoding them in source code.
typescript, ts-node, @types/node	Lets us write and run <code>.ts</code> files directly with <code>npx ts-node</code> , without a separate compile step.

Step 3 — Initialize TypeScript

```
npx tsc --init
```

This creates a `tsconfig.json`. You don't need to touch it for this guide — the defaults are fine for running scripts with `ts-node`.

Step 4 — Create your `.env` file

In the project root, create a file named `.env`:

```
OPENAI_API_KEY=sk-your-key-here
TAVILY_API_KEY=tvly-your-key-here
```

- Get an OpenAI key from the OpenAI platform dashboard (you'll need a small amount of prepaid credit — these labs cost fractions of a cent each with `gpt-4o-mini`).
- Get a free Tavily key from tavily.com — you only need this from Chapter 5 onward, so feel free to skip it until then.

Common mistake: committing `.env` to git. Create a `.gitignore` file containing `.env` and `node_modules` right now, before you forget.

Step 5 — Verify your setup

Create `00-setup-check.ts`:

```
import "dotenv/config";
import { ChatOpenAI } from "@langchain/openai";

async function main() {
  const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });
  const response = await model.invoke("Say 'setup successful' and nothing else.");
  console.log(response.content);
}

main();
```

Run it:

```
npx ts-node 00-setup-check.ts
```

Expected result: the console prints `setup successful` (or very close to it).

How to verify it worked: if you see that text with no errors, your API key, packages, and TypeScript config are all correctly wired together. If you see an authentication error, double check there are no extra spaces or quotes around your key in `.env`.

You're now ready for Chapter 1.

Chapter 1: From Chains to Agents — What Makes an Agent Different

1.1 Concept and Theory

The analogy first. Imagine you hire two assistants.

Assistant A follows a script you wrote: "Open the mailbox, read the letter, type a summary, send it to me." No matter what's in the letter, Assistant A does these four steps, in this order, every time. This is a **Chain**. You, the developer, decided the entire sequence of steps in advance.

Assistant B is different. You just say: "Handle whatever comes up today, and here's a phone, a calculator, and a filing cabinet you're allowed to use." Assistant B looks at each situation, decides *which* tool (if any) is needed, uses it, looks at the result, and decides what to do next — repeating until the task is done. This is an **Agent**.

Why this distinction matters. A Chain is predictable but rigid: it cannot adapt its own steps based on what it discovers along the way. A chain that says "retrieve documents, then answer" will always retrieve documents — even for a question like "what's 2+2?" where retrieval is pointless. An Agent, by contrast, decides *at runtime* whether it needs a tool at all, and if so, which one.

What an Agent actually is, technically. An agent is a loop built around a language model. On each turn of the loop, the model is given:

- the original question,
- a list of tools it's allowed to use (each with a name and description),

- everything that's happened so far in this loop (which tools were called, and what they returned).

The model then outputs one of two things: "call this tool, with this input" or "here is my final answer." If it asks for a tool, the agent framework actually runs that tool, feeds the result back to the model, and loops again. If it gives a final answer, the loop stops.

When to use an agent vs. a chain. Use a Chain when you already know the exact sequence of steps a task requires (e.g., "translate, then summarize" is always two fixed steps). Use an Agent when the number and order of steps depends on the specific input — for example, "answer this question" might need zero tool calls, one tool call, or five, depending on what's asked.

A note on terminology you'll see in docs. "AgentExecutor" is the piece of LangChain.js that actually *runs* the loop described above. The "agent" itself (created by `createReactAgent` or `createToolCallingAgent`, which you'll meet in the next two chapters) is just the logic for *one turn* of deciding what to do next. `AgentExecutor` is what repeats that turn-logic until a final answer appears.

1.2 Practical Lab: Feeling the Limitation of a Chain

There's no agent code yet in this lab — the point is to *feel* the problem an agent solves, using only what you already know (a simple chain).

Step 1 — Create `01-chain-limitation.ts`

```
import "dotenv/config";
import { ChatOpenAI } from "@langchain/openai";
import { PromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";

async function main() {
  const model = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });

  // A fixed, hardcoded sequence: ALWAYS do math "by hand" via the LLM.
  const prompt = PromptTemplate.fromTemplate(
    "You are a calculator. Compute this expression and return ONLY the number: {expression}"
  );
  const chain = prompt.pipe(model).pipe(new StringOutputParser());

  const result = await chain.invoke({
    expression: "47.5 * 23.8 + 19234 / 17",
  });

  console.log("Chain's answer:", result);
  console.log("Actual answer:", 47.5 * 23.8 + 19234 / 17);
}

main();
```

Why we wrote it this way: this chain has exactly one step — ask the LLM to compute an expression as if it were a calculator. There is no way for this chain to verify its own arithmetic; it can only generate a plausible-looking number.

Step 2 — Run it

```
npx ts-node 01-chain-limitation.ts
```

Expected result: the chain's answer will likely be *close* to the actual answer, but probably not exactly equal. Large language models are pattern-predictors, not calculators — they're surprisingly bad at exact multi-step arithmetic, especially with decimals.

How to verify it worked correctly (or didn't): compare the two printed numbers. A mismatch isn't a bug in your code — it's the actual limitation we're illustrating. Try a couple of different expressions and watch how the size of the error tends to grow with the complexity of the expression.

The takeaway to sit with: the chain *cannot* decide "this sub-task needs a real calculator, not a guess." It has no mechanism to reach for a tool. That missing mechanism is exactly what an agent adds, starting next chapter.

1.3 Knowledge Check

Summary: A Chain executes a fixed, developer-defined sequence of steps every time. An Agent is a loop in which a language model decides, step by step, whether to call a tool, which tool to call, and when it has enough information to stop. `AgentExecutor` is the `LangChain.js` component that runs this loop until a final answer is produced.

Key takeaways:

- Chains are predictable and simple but cannot adapt their steps to the input.
- Agents trade some predictability for the ability to dynamically choose actions.
- An agent's "intelligence" comes entirely from the underlying LLM deciding, at each turn, what to do next.
- Tool calls happen *outside* the LLM — the LLM only decides to ask for them; your code (via `AgentExecutor`) actually executes them.

Practice questions:

1. In your own words, what is the one capability a Chain lacks that an Agent has?
2. Why did the chain in the lab struggle with arithmetic, even though it's "an AI"?
3. Give one real-world task you'd implement as a Chain, and one you'd implement as an Agent. Justify each choice.

Mini exercise: Modify `01-chain-limitation.ts` to try five different arithmetic expressions of increasing complexity (e.g., a 2-number addition, then a 5-number expression with decimals and division). Note at what complexity level the chain's answers start being wrong. Keep this file — you'll fix this exact problem with a real tool in Chapter 3.

Chapter 2: The ReAct Pattern — Reasoning + Acting

2.1 Concept and Theory

The analogy. Think about how you solve a tricky problem out loud: "*Hmm, I think I need the population of France first... let me look that up... okay, got it, 68 million... now I need to divide that*

by the number of regions... let me calculate that..." You alternate between **thinking** (reasoning about what you need) and **doing** (taking an action to get information), using each result to decide your next thought. That alternating pattern — Think, Act, Observe, Think, Act, Observe... — is called **ReAct** (a blend of "Reasoning" and "Acting").

Why ReAct exists. Early on, researchers found that if you just ask a model "Use these tools to answer this question," it tends to either over-use tools unnecessarily or jump to a wrong answer without checking anything. ReAct fixes this by forcing the model to externalize its reasoning in a structured, repeating format, so it explicitly reasons about *why* it's about to take an action before taking it — and explicitly reasons about *what it learned* before deciding what to do next.

What the loop actually looks like as text. When you use a classic ReAct agent, behind the scenes the model is shown a prompt that looks roughly like this, and it continues the pattern:

```
Question: What is 23 times 4, plus the square root of 49?
Thought: I should calculate 23 times 4 first.
Action: calculator
Action Input: 23 * 4
Observation: 92
Thought: Now I need the square root of 49.
Action: calculator
Action Input: sqrt(49)
Observation: 7
Thought: I now have both numbers and can add them.
Action: calculator
Action Input: 92 + 7
Observation: 99
Thought: I now know the final answer.
Final Answer: 99
```

Each **Thought** is the model reasoning. Each **Action + Action Input** is the model asking your code to run a specific tool with specific input. Each **Observation** is the *actual* result your code got back from running that tool — fed back into the model's next turn. The loop ends the moment the model writes **Final Answer:** instead of another **Action:**.

Where the prompt comes from. Rather than writing this entire instructional template yourself, LangChain provides a ready-made, well-tested version of it on "LangChain Hub" — a shared library of prompts. We'll pull the most widely used one, named `hwchase17/react`.

An important nuance: classic ReAct (what you'll build in this lab) relies on the model correctly *formatting its text* in this exact pattern. Most older or smaller LLMs were only trained loosely on this format, which is why parsing errors happen (covered in Chapter 6). Modern OpenAI models support a more reliable mechanism called **native tool calling**, which we'll switch to in Chapter 4. We're starting with classic ReAct first because it makes the reasoning loop fully visible and easy to understand — you're seeing the agent "think out loud" in plain text.

2.2 Practical Lab: Your First ReAct Agent

Step 1 — Create a calculator tool (a minimal version; we'll do this properly in Chapter 3)

Create `02-react-agent.ts`:

```
import "dotenv/config";
import { z } from "zod";
import { tool } from "@langchain/core/tools";
```

```

import { OpenAI } from "@langchain/openai";
import { pull } from "langchain/hub";
import { createReactAgent, AgentExecutor } from "langchain/agents";
import type { PromptTemplate } from "@langchain/core/prompts";

// A simple calculator tool. We'll explain `tool()` properly in Chapter 3 –
// for now, just notice it has a name, a description, and a function.
const calculatorTool = tool(
  async ({ expression }) => {
    // WARNING: eval() is used here ONLY for this teaching example.
    // Never eval() untrusted input in a real application – see Chapter 3.
    return String(eval(expression));
  },
  {
    name: "calculator",
    description:
      "Evaluates a basic arithmetic expression like '23 * 4' or '92 + 7'. Input must be a valid JavaScript math expression.",
    schema: z.object({
      expression: z.string().describe("A math expression to evaluate"),
    }),
  }
);

async function main() {
  // classic ReAct prompting expects a plain completion-style model
  const llm = new OpenAI({ model: "gpt-4o-mini", temperature: 0 });

  // Pull the standard, battle-tested ReAct prompt from LangChain Hub
  const prompt = await pull<PromptTemplate>("hwchase17/react");

  const tools = [calculatorTool];

  const agent = await createReactAgent({ llm, tools, prompt });

  const agentExecutor = new AgentExecutor({
    agent,
    tools,
    verbose: true, // <-- this prints every Thought/Action/Observation to your console
  });

  const result = await agentExecutor.invoke({
    input: "What is 23 times 4, plus the square root of 49?",
  });

  console.log("\nFINAL ANSWER:", result.output);
}

main();

```

Why each piece is there:

- `tool( ... )` packages a plain async function into something the agent framework can call — note the `name` and `description`, which is literally how the model decides *when* to reach for this tool. We cover this deeply in Chapter 3.
- `pull("hwchase17/react")` downloads the standard ReAct prompt template so you don't have to write the Thought/Action/Observation instructions yourself.
- `createReactAgent({ llm, tools, prompt })` combines the model, the tools,

and the prompt into "one turn" of agent logic.

- `new AgentExecutor({ agent, tools, verbose: true })` wraps that one-turn logic into the repeating loop, and `verbose: true` is what lets you *see* the reasoning happen, instead of only getting the final answer.

Step 2 — Run it

```
npx ts-node 02-react-agent.ts
```

Expected result: because `verbose: true` is set, your terminal will print a colored trace showing each `Thought`, `Action`, `Action Input`, and `Observation`, ending with `Final Answer: 99`. Your own `console.log` then prints that same final answer cleanly at the bottom.

How to verify it worked correctly: check that the printed `Observation` values match real arithmetic (92, then 7, then 99) — and that the final answer is mathematically correct, unlike Chapter 1's chain.

Common mistakes and fixes:

- *"Cannot find module 'langchain/hub'"* — make sure you installed the base `langchain` package (not just `@langchain/core`), since `pull` lives there.
- *Authentication / network error on pull(...)* — `pull` fetches the prompt from LangChain's hosted hub over the internet the first time; make sure you have an internet connection. (No API key is required for pulling public prompts.)
- *The agent loops strangely or repeats itself* — this usually means the model's text output didn't perfectly match the expected `Action: / Action Input:` format. We tackle this exact failure mode head-on in Chapter 6.
- *Using eval() made you uneasy* — good instinct! That's intentional foreshadowing for Chapter 3, where we replace it with `safe`, explicit math operations.

2.3 Knowledge Check

Summary: ReAct is a prompting pattern that has the model alternate between reasoning ("Thought"), acting ("Action" + "Action Input"), and observing results ("Observation"), repeating until it's confident enough to give a "Final Answer." `createReactAgent` builds one turn of this loop using a prompt pulled from LangChain Hub; `AgentExecutor` repeats it until completion.

Key takeaways:

- The model's "Thought" is just text it generates — its visible reasoning, not a separate hidden process.
- "Observation" is *real* data your code retrieved by actually running the requested tool — the model never "imagines" this part.
- `verbose: true` is one of the most useful debugging settings you'll use throughout this whole chapter of your learning.
- Classic ReAct depends on the model reliably producing exact text formatting, which is a source of fragility we'll address later.

Practice questions:

1. What is the difference between a "Thought" and an "Observation" in a ReAct trace?
2. Why does the loop stop exactly when the model writes "Final Answer:"?
3. What would happen (in theory) if the calculator tool returned the wrong result for one step — would the agent "notice"?

Mini exercise: Change the question passed to `agentExecutor.invoke(...)` to something that requires three sequential calculations (e.g., "What is (15 + 9) times 3, divided by 4?"). Watch the verbose trace and count how many Thought/Action/Observation cycles it takes.

Chapter 3: Defining Tools — Teaching the Agent New Skills

3.1 Concept and Theory

The analogy. A tool, to an LLM, is like a labeled button on a remote control it's never seen before. It can't see the wiring behind the button — it only sees the label ("Volume Up") and maybe a tiny printed instruction ("press to increase speaker volume by 1 unit"). Based purely on that label and instruction, it decides *whether* and *when* to press it. If the label is vague ("Button 7"), the model will misuse it or ignore it. If the label is precise ("calculator: evaluates one arithmetic expression and returns a number"), the model uses it confidently and correctly.

The three things every tool needs, and why:

1. **A name.** This is the literal identifier the model writes when it wants to use the tool (e.g., `Action: calculator`). It should be short, and ideally written in `snake_case` with no spaces — some model providers handle spaces or punctuation in tool names poorly.
2. **A description.** This is the *single most important* part of a tool definition. The model never sees your function's source code — it only sees this description. A vague description ("does math stuff") leads to a model that either never uses the tool or uses it for the wrong situations. A precise description ("Evaluates a single arithmetic expression containing only numbers and + - * / () and returns the numeric result") leads to confident, correct usage.
3. **A schema.** This describes the *shape* of the input the tool expects — not as English text, but as a strict, machine-checkable structure (we use Zod for this). The schema is what lets LangChain (and the model, behind the scenes) know exactly which fields to provide, what type each one is, and even a per-field description that further hints at meaning.

Why Zod, specifically. Zod is a TypeScript-first validation library: you write `z.object({ a: z.number(), b: z.number() })` and you get both (a) a runtime check that incoming data really is `{ a: number, b: number }`, and (b) full TypeScript autocomplete and type-checking inside your tool's function body. LangChain.js converts your Zod schema into a JSON Schema under the hood, which is the format model providers like OpenAI actually expect when listing available tools.

The function itself. This is ordinary TypeScript/JavaScript — synchronous or `async` — that receives the validated input object and returns a result (usually a string, since that's easiest for the

model to read back). This is where you do real work: call an API, compute something, query a database, read a file. The model never executes this code directly; your `AgentExecutor` does, and only passes the *result text* back to the model.

Safety note (the `eval()` we used in Chapter 2): `eval()` executes arbitrary code, including malicious code. We used it briefly purely so the lab had as little new syntax as possible while you focused on the ReAct loop. From this chapter onward, we replace it with an explicit, safe set of math operations.

3.2 Practical Lab: Building Two Real Custom Tools

We'll build a **safe calculator tool** (fixing Chapter 2's `eval()` shortcut) and a brand-new **text analysis tool**, to see tools applied to a non-math problem too.

Step 1 — Create `03-custom-tools.ts`

```
import "dotenv/config";
import { z } from "zod";
import { tool } from "@langchain/core/tools";

// ---- Tool 1: a SAFE calculator (no eval!) ----
// We restrict it to one operation at a time, with explicit operands.
// This is more verbose than `eval()`, but it can never execute arbitrary code.
const calculatorTool = tool(
  async ({ operation, a, b }) => {
    switch (operation) {
      case "add":
        return String(a + b);
      case "subtract":
        return String(a - b);
      case "multiply":
        return String(a * b);
      case "divide":
        if (b === 0) return "Error: cannot divide by zero.";
        return String(a / b);
      default:
        return `Error: unknown operation "${operation}"`;
    }
  },
  {
    name: "calculator",
    description:
      "Performs one arithmetic operation (add, subtract, multiply, or divide) on two numbers. Call this once per individual operation – for multi-step math, call it multiple times in sequence.",
    schema: z.object({
      operation: z
        .enum(["add", "subtract", "multiply", "divide"])
        .describe("Which operation to perform"),
      a: z.number().describe("The first number"),
      b: z.number().describe("The second number"),
    })
  }
);

// ---- Tool 2: a text analysis tool (shows tools aren't just for math) ----
const textStatsTool = tool(
  async ({ text }) => {
    const wordCount = text.trim().split(/\s+/).filter(Boolean).length;
    const charCount = text.length;
  }
);
```

```

    const sentenceCount = (text.match(/[.!?]+/g) || []).length;
    return JSON.stringify({ wordCount, charCount, sentenceCount });
  },
  {
    name: "text_stats",
    description:
      "Analyzes a block of text and returns its word count, character count, and sentence count as JSON.",
    schema: z.object({
      text: z.string().describe("The text to analyze"),
    }),
  }
);

async function main() {
  // Before plugging tools into an agent, ALWAYS test them directly first.
  // This isolates bugs: is the problem in the tool, or in the agent's usage of it?
  console.log("Direct calculator test:");
  console.log(
    await calculatorTool.invoke({ operation: "multiply", a: 23, b: 4 })
  );

  console.log("\nDirect text_stats test:");
  console.log(
    await textStatsTool.invoke({
      text: "LangChain makes it easier to build LLM apps. Agents are powerful.",
    })
  );
}

main();

```

Why we test tools directly, with `.invoke()`, before involving any agent: every `tool()` you create is itself a small Runnable — it has its own `.invoke()` method, completely independent of any agent. Calling it directly lets you confirm the tool's logic is correct *in isolation*, the same way you'd unit-test a function before wiring it into a larger system. If something's wrong later inside an agent, you'll already know the tool itself isn't the cause.

Step 2 — Run it

```
npx ts-node 03-custom-tools.ts
```

Expected result:

```

Direct calculator test:
92

Direct text_stats test:
{"wordCount":10,"charCount":67,"sentenceCount":2}
````//
(Your exact `charCount` may differ slightly depending on exact spacing – that's fine.)

How to verify it worked correctly: the calculator output should match `23 * 4 = 92` exactly. The `text_stats` output should be valid JSON with three numeric fields – paste it into any JSON validator if you want to double-check the formatting.

Common mistakes and fixes:
- *TypeScript complains about `a` and `b` being `unknown`* – this means the Zod

```

schema isn't matching your function's destructured parameters. Double-check that your function's parameter names (`{ operation, a, b }`) exactly match the keys in your `z.object({...})``.

- *\*Forgetting `.describe()` on individual fields\** – your tool will still *\*work\**, but the model will have less context for filling in that field correctly on tricky inputs. Always add a short `.describe()` per field; it's cheap and meaningfully improves reliability.

- *\*Returning a raw object instead of a string from a tool function\** – LangChain.js usually stringifies it for you, but it's best practice to explicitly `JSON.stringify(...)` (as we did) so you control exactly what the model sees.

### ### 3.3 Knowledge Check

**\*\*Summary:\*\*** A LangChain tool is a small, self-contained unit made of three parts: a `name` the model references, a `description` the model reads to decide when to use it, and a `schema` (built with Zod) that defines exactly what input it accepts. The function body is ordinary code that does the real work and returns a result. You should always test a tool directly with `.invoke()` before wiring it into any agent.

**\*\*Key takeaways:\*\***

- The `description` is doing more work than the function name – it's the model's *\*only\** window into what the tool is for.
- Strict schemas (`z.enum`, `z.number`, etc.) catch malformed input before your function body ever runs.
- Never use `eval()` or similarly unrestricted code execution inside a real tool.
- Testing a tool with `.invoke()` is the equivalent of unit-testing – isolate the tool from the agent when debugging.

**\*\*Practice questions:\*\***

1. Why is a vague tool description (e.g., "does stuff with numbers") worse than no description at all?
2. What's the practical difference between `z.number()` and `z.string()` for an input the agent might supply as `"23"` vs `23`?
3. Why is `eval()` dangerous specifically in the context of an LLM-driven tool, as opposed to a regular script you wrote yourself?

**\*\*Mini exercise:\*\*** Add a third tool, `wordReverser`, that takes a single word and returns it spelled backwards. Give it a precise name, description, and Zod schema, then test it directly with `.invoke()` before moving on – don't wire it into an agent yet, that's coming in Chapter 7.

---

## ## Chapter 4: Tool-Calling Agents – The Modern, Native Approach

### ### 4.1 Concept and Theory

**\*\*The analogy, continued.\*\*** In Chapter 2, our model had to *\*write out\**, in plain English-like text, something resembling `Action: calculator\nAction Input: {"operation": "multiply", "a": 23, "b": 4}` – and our code had to carefully *\*parse\** that text back into real data. It's like asking someone to fill out a form by handwriting it in a very specific paragraph style, then having someone else try to read the handwriting and re-type it into a spreadsheet. It mostly works, but handwriting (text formatting) can be misread.

**\*\*What changed.\*\*** Major model providers, including OpenAI, eventually built *\*native tool calling\** directly into their APIs. Instead of describing tools in plain text inside the prompt and parsing the model's text response, you send the tool definitions as *\*structured data\** (the JSON Schema we mentioned in Chapter 3), and the model returns its tool-call request as *\*structured data\** too – no parsing of free-form text required. It's the difference between handwriting a

form and filling out a real digital form with dropdowns and fixed fields: far fewer ways to get it wrong.

**\*\*Why this matters practically.\*\*** Native tool calling is more reliable (fewer malformed outputs), often faster, and supports calling *multiple tools* in a single turn\* if the model decides several are needed at once (e.g., "look up the weather AND convert currency" in one step) – something classic text-based ReAct struggles to do cleanly.

**\*\*The new building block: `createToolCallingAgent`.** This is almost a drop-in replacement for `createReactAgent` from Chapter 2, but instead of using a `pull`-ed text-based ReAct prompt, you build your own short `ChatPromptTemplate` with two required placeholders:

- `{agent\_scratchpad}` – where the framework injects the history of tool calls and results so far in this turn's reasoning.
- `{input}` – the user's actual question.

You can optionally add a `{chat\_history}` placeholder too, to support multi-turn conversations (more on that when you get to your Memory unit, which you've already covered).

**\*\*A quick mental model for choosing between Chapter 2's and Chapter 4's approach:\*\*** if your model supports native tool calling (essentially all current OpenAI chat models do), prefer `createToolCallingAgent`. Classic `createReactAgent` is mainly useful for older or smaller models without native tool-calling support, or for educational visibility into the raw text-based reasoning loop, which is exactly why we taught it first.

#### ### 4.2 Practical Lab: Rebuilding the Calculator Agent, the Modern Way

We'll reuse the safe `calculatorTool` from Chapter 3 and rebuild the exact same agent behavior from Chapter 2 – but using native tool calling.

**\*\*Step 1 – Create `04-tool-calling-agent.ts`\*\***

```
``typescript
import "dotenv/config";
import { z } from "zod";
import { tool } from "@langchain/core/tools";
import { ChatOpenAI } from "@langchain/openai";
import { ChatPromptTemplate, MessagesPlaceholder } from
"@langchain/core/prompts";
import { createToolCallingAgent, AgentExecutor } from "langchain/agents";

const calculatorTool = tool(
 async ({ operation, a, b }) => {
 switch (operation) {
 case "add":
 return String(a + b);
 case "subtract":
 return String(a - b);
 case "multiply":
 return String(a * b);
 case "divide":
 if (b === 0) return "Error: cannot divide by zero.";
 return String(a / b);
 default:
 return `Error: unknown operation "${operation}".`;
 }
 },
 {
 name: "calculator",
 description:
 "Performs one arithmetic operation (add, subtract, multiply, or divide) on
```

```

two numbers.",
 schema: z.object({
 operation: z.enum(["add", "subtract", "multiply", "divide"]),
 a: z.number(),
 b: z.number(),
 }),
},
);

async function main() {
 // Notice: ChatOpenAI (chat-style), not OpenAI (completion-style) like Chapter
2.
 // Native tool calling is a feature of chat models specifically.
 const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });

 const prompt = ChatPromptTemplate.fromMessages([
 ["system", "You are a helpful assistant that uses tools when needed to
answer accurately."],
 ["human", "{input}"],
 new MessagesPlaceholder("agent_scratchpad"),
]);

 const tools = [calculatorTool];

 const agent = createToolCallingAgent({ llm, tools, prompt });

 const agentExecutor = new AgentExecutor({
 agent,
 tools,
 verbose: true,
 });

 const result = await agentExecutor.invoke({
 input: "What is 23 times 4, plus the square root of 49?",
 });

 console.log("\nFINAL ANSWER:", result.output);
}

main();

```

### Why each change matters, compared to Chapter 2:

- `ChatOpenAI` instead of `OpenAI`: native tool calling is exposed through the chat-completion API, not the older plain-text completion API.
- `ChatPromptTemplate` with `{input}` and `agent_scratchpad`: there's no giant ReAct instruction block here — the model doesn't need to be *told* how to format a tool call in English, because tool calling is a built-in capability of the model itself, not a text convention.
- `createToolCallingAgent` instead of `createReactAgent`: same overall job (build one turn of agent reasoning), different underlying mechanism (structured function calls instead of parsed text).

### Step 2 — Run it

```
npx ts-node 04-tool-calling-agent.ts
```

**Expected result:** the verbose trace now looks noticeably different from Chapter 2's — instead of

**Thought:/Action:** text blocks, you'll see structured tool-call entries showing the exact JSON arguments sent to `calculator`. The final printed answer should still be mathematically correct (the model will likely call the calculator three times:  $23 \times 4$ , note the square root of 49 is a known value it may compute directly or via tool, then add).

**How to verify it worked correctly:** confirm the final answer is 99, same as Chapter 2 — we're testing a different *mechanism*, not a different *result*. If your tool only supports the four basic operations, watch how the model handles "square root" — it may answer it directly from its own knowledge (49's square root is well-known) rather than calling a tool for it, since you didn't give it a square-root tool. This is expected and actually a good sign of sensible behavior, not a bug.

#### Common mistakes and fixes:

- *Forgetting `agent_scratchpad` as a `MessagesPlaceholder`* — you'll get an error about a missing input variable. Both `createReactAgent` and `createToolCallingAgent` require this exact placeholder somewhere in the prompt; it's how intermediate steps get communicated back to the model.
- *Using `OpenAI` instead of `ChatOpenAI`* — `createToolCallingAgent` requires a model that supports the chat-style tool-calling interface; the plain completion-style `OpenAI` class will throw an error.
- *Expecting identical verbose output to Chapter 2* — it will look different by design; focus on whether the *final answer* is correct, not on matching the trace format.

## 4.3 Knowledge Check

**Summary:** Native tool calling lets the model request a tool call as structured data instead of formatted text, making agent behavior more reliable. `createToolCallingAgent` builds one turn of this style of reasoning, used alongside a `ChatPromptTemplate` containing an `{input}` and an `agent_scratchpad` placeholder, and is the recommended default for any model that supports it (which includes virtually all current OpenAI chat models).

#### Key takeaways:

- Native tool calling = structured request/response, not text parsing.
- `ChatOpenAI` (chat model) is required for `createToolCallingAgent`; `OpenAI` (completion model) was required for the older `createReactAgent`.
- The required prompt structure is much shorter than the classic ReAct prompt, because the format itself is built into the model's API.
- Same `AgentExecutor` wraps both styles of agent — the loop-running mechanism didn't change, only how each turn's "decide what to do" logic works internally.

#### Practice questions:

1. What concrete failure mode (seen in Chapter 2's common mistakes) does native tool calling reduce?
2. Why must `ChatOpenAI` be used here instead of `OpenAI`?

3. Looking at your two prompt templates (Chapter 2's pulled ReAct prompt vs. Chapter 4's short ChatPromptTemplate) — why is the second one so much shorter?

**Mini exercise:** Add the `textStatsTool` from Chapter 3 into this agent's `tools` array (alongside `calculatorTool`) and ask it: *"How many words are in the sentence 'LangChain agents are genuinely useful', and also what's 12 divided by 4?"* Watch the verbose trace to see it pick both tools correctly within one run.

---

## Chapter 5: Built-in / Real-World Tools — Giving Your Agent a Window to the World

### 5.1 Concept and Theory

**The analogy.** Every tool so far has been something we wrote ourselves, running entirely on your machine, using information your machine already has. But a huge category of useful agent tasks needs information that *doesn't exist anywhere on your machine* — like "what happened in the news today?" or "what's the current price of something?" A calculator that can only do arithmetic on numbers you already know is useful, but an assistant that can also *go look something up* is far more capable. That's what a **search tool** gives an agent: a window out into the live web.

**Why this is just "another tool," conceptually.** This is the most important idea in this chapter: a web search tool is *not* architecturally special. It has a **name**, a **description**, and a **schema**, exactly like your hand-written tools from Chapter 3 — the only difference is that its function body, instead of computing something locally, makes an HTTP request to a search API and returns the results as text. From the agent's perspective, calling a search tool and calling your `calculator` tool feel identical: "ask for something, get text back."

**Why we use Tavily specifically.** Tavily is a search API purpose-built for LLM agents — its results come back as clean, model-friendly summaries rather than raw, messy HTML, which means less post-processing work for you and clearer information for the model to reason over. LangChain.js ships an official, ready-made tool wrapper for it (`@langchain/tavily`), so you don't need to write your own search-tool wrapper from scratch — though conceptually, you absolutely could, the same way you built `calculator` and `text_stats`.

**Why agents need a search tool at all, philosophically.** A language model's knowledge is frozen at whatever point its training data ended. It cannot know about anything that happened afterward, and it has no live connection to the world by default. Giving it a search tool effectively patches that gap — letting it reason about things it was never trained on, by *fetching* the relevant facts at the moment it needs them, the same way you'd open a browser tab mid-conversation to check something.

### 5.2 Practical Lab: Adding Web Search to Your Agent

**Step 1 — Make sure your `.env` has a Tavily key**

```
TAVILY_API_KEY=tvly-your-key-here
```

(Sign up free at [tavily.com](https://tavily.com) if you skipped this in Chapter 0.)

**Step 2 — Create `05-web-search-agent.ts`**



```

import "dotenv/config";
import { ChatOpenAI } from "@langchain/openai";
import { TavilySearch } from "@langchain/tavily";
import { ChatPromptTemplate, MessagesPlaceholder } from
"@langchain/core/prompts";
import { createToolCallingAgent, AgentExecutor } from "langchain/agents";

async function main() {
 const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });

 // A ready-made tool – notice it STILL has a name and description under the
hood,
// we just didn't have to write them ourselves this time.
 const searchTool = new TavilySearch({ maxResults: 3 });

 const prompt = ChatPromptTemplate.fromMessages([
 [
 "system",
 "You are a helpful research assistant. Use the search tool for questions
about current events, recent facts, or anything you're not certain about.",
],
 ["human", "{input}"],
 new MessagesPlaceholder("agent_scratchpad"),
]);

 const tools = [searchTool];

 const agent = createToolCallingAgent({ llm, tools, prompt });
 const agentExecutor = new AgentExecutor({ agent, tools, verbose: true });

 const result = await agentExecutor.invoke({
 input: "What is LangChain, and has it released any major updates recently?",
 });

 console.log("\nFINAL ANSWER:", result.output);
}

main();

```

**Why we phrased the system message this way:** explicitly telling the model *when* to reach for search ("current events, recent facts, anything you're not certain about") reduces wasted search calls on questions the model can already answer confidently from its own training — search isn't free, and unnecessary calls add latency and cost.

### Step 3 — Run it

```
npx ts-node 05-web-search-agent.ts
```

**Expected result:** the verbose trace shows the model calling `tavily_search` (or similarly named) with a query it generated itself (e.g., "LangChain recent updates 2026"), receiving back a handful of search result snippets, and synthesizing a final answer that references genuinely current information.

**How to verify it worked correctly:** the final answer should mention specifics (version numbers, recent dates, feature names) that a model's frozen training data alone couldn't reliably produce — that's your signal the search tool was actually used and actually helped, rather than the model just guessing.

**Common mistakes and fixes:**

- *"Missing TAVILY\_API\_KEY" error* — confirm `.env` has the exact variable name `TAVILY_API_KEY` and that you called `import "dotenv/config";` at the top of the file before anything else.
- *Agent answers without searching at all* — try a more clearly time-sensitive question ("what happened in AI news this week?") — for very generic questions, a well-instructed model may correctly decide search isn't necessary, which is actually correct behavior, not a bug.
- *Search results look irrelevant* — try lowering `maxResults` noise by being more specific in your `input` question; vague questions produce vague search queries.

## 5.3 Knowledge Check

**Summary:** A web search tool is structurally identical to any custom tool you've built — it just happens to fetch live information from the internet rather than compute something locally. `@langchain/tavily`'s `TavilySearch` provides a ready-made search tool, letting an agent answer questions about information outside its training data by fetching it at the moment it's needed.

### Key takeaways:

- Built-in tools follow the exact same `name/description/schema` contract as your hand-written tools.
- Giving an agent a search tool patches the "frozen knowledge" limitation inherent to all LLMs.
- A good system prompt should hint at *when* to use search, to avoid wasteful or unnecessary calls.
- You can always verify search actually happened by checking the verbose trace for a tool call, rather than assuming.

### Practice questions:

1. Why is a search tool "just another tool" architecturally, rather than something fundamentally different from `calculator`?
2. What real limitation of LLMs does a search tool address?
3. What's one risk of giving an agent unrestricted access to search every single question, even simple ones?

**Mini exercise:** Combine `searchTool` and `calculatorTool` (from Chapter 4) into one agent's `tools` array, and ask: *"Search for the current population of Japan, then divide that number by 50."* Watch the trace to see it chain a search result into a calculation — this is a preview of Chapter 7.

---

## Chapter 6: Controlling the Loop — Stopping Conditions, Max Iterations & Error Handling

### 6.1 Concept and Theory

**The analogy.** Imagine giving Assistant B (from Chapter 1) a task with a flawed or ambiguous tool, and no instruction on when to give up. They might call the same tool over and over, hoping for a different result, burning your time (and your API budget) indefinitely. A good manager sets boundaries: "if you've tried five different approaches and still haven't solved it, stop and report back what you found." That's exactly what stopping conditions do for an agent.

**Why an unbounded loop is dangerous in practice.** Each turn of the agent loop is a real API call to a language model, which costs money and time. If a tool's description is ambiguous, or a tool returns an error the model misinterprets, the model can get stuck calling the same tool repeatedly, theoretically forever. Without a hard limit, a single confused agent run could rack up an enormous bill or simply never return.

**The three knobs you actually have:**

1. **maxIterations** — a hard cap on the number of Thought/Action/Observation (or tool-call) cycles `AgentExecutor` will run before forcibly stopping and returning whatever it has, even if it never produced a clean "Final Answer." Think of this as "try at most N times, then give up gracefully."
2. **maxExecutionTime** — a wall-clock timeout, in milliseconds, independent of how many iterations happened. Useful when individual tool calls (like a slow API) might each take a long time, so counting iterations alone isn't a tight enough safety net.
3. **handleParsingErrors** — specifically relevant to *classic* ReAct agents (Chapter 2's style), where the model's raw text sometimes doesn't perfectly match the expected `Action: / Action Input: format`. Setting this to `true` tells `AgentExecutor` to catch that malformed output, feed a corrective message back to the model ("your last output couldn't be parsed, please follow the format"), and let it try again — rather than crashing your whole program on one bad text generation.

**A useful diagnostic option:** `returnIntermediateSteps: true` tells `AgentExecutor` to include the full list of every tool call and result it made along the way in its final returned object — not just the final answer. This is invaluable for debugging *why* an agent reached a particular conclusion, separate from the live `verbose` console trace.

### 6.2 Practical Lab: Triggering and Taming Failure Modes

We'll deliberately create a tool with a misleading description to provoke confused looping, then apply the safety knobs above to control it.

#### Step 1 — Create `06-stopping-conditions.ts`

```
import "dotenv/config";
import { z } from "zod";
import { tool } from "@langchain/core/tools";
import { ChatOpenAI } from "@langchain/openai";
import { ChatPromptTemplate, MessagesPlaceholder } from
```

```

"@langchain/core/prompts";
import { createToolCallingAgent, AgentExecutor } from "langchain/agents";

// Deliberately unhelpful: the description doesn't say HOW to fix a bad result,
// so a model can plausibly retry it pointlessly on certain inputs.
const flakyLookupTool = tool(
 async ({ id }) => {
 // Always "fails" in a way that doesn't explain what a correct ID looks
 like.
 return `Error: no record found for id "${id}".`;
 },
 {
 name: "record_lookup",
 description: "Looks up a record by ID.",
 schema: z.object({ id: z.string() }),
 }
);

async function main() {
 const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });

 const prompt = ChatPromptTemplate.fromMessages([
 ["system", "You are a helpful assistant. Use record_lookup to find the requested record."],
 ["human", "{input}"],
 new MessagesPlaceholder("agent_scratchpad"),
]);

 const tools = [flakyLookupTool];
 const agent = createToolCallingAgent({ llm, tools, prompt });

 const agentExecutor = new AgentExecutor({
 agent,
 tools,
 verbose: true,
 maxIterations: 3, // hard stop after 3 cycles
 maxExecutionTime: 20000, // 20-second wall-clock safety net
 returnIntermediateSteps: true,
 });

 const result = await agentExecutor.invoke({
 input: "Look up the record for id 'ABC-123' and tell me what's in it.",
 });

 console.log("\nFINAL OUTPUT:", result.output);
 console.log("\nNUMBER OF TOOL CALLS MADE:", result.intermediateSteps.length);
}

main();

```

**Why we set `maxIterations: 3` deliberately low here:** it's low enough to clearly demonstrate the cutoff behavior within one quick run, rather than waiting through dozens of cycles. In a real application, you'd typically use a more generous value (the LangChain.js default is 15) and tune it based on how complex your real tasks are.

## Step 2 — Run it

```
npx ts-node 06-stopping-conditions.ts
```

**Expected result:** the verbose trace shows the model trying `record_lookup` (possibly more than once, perhaps with slightly different formatting of the ID), getting the same "no record found" error

each time, and — because we capped it at 3 iterations — `AgentExecutor` forcibly stops and returns its best-effort final output (likely something like "I was unable to find a record with that ID") instead of looping forever.

**How to verify it worked correctly:** `NUMBER OF TOOL CALLS MADE` should be at most 3 (matching `maxIterations`), proving the cap was actually enforced rather than just documented.

### Step 3 — Now try a classic ReAct agent with `handleParsingErrors`

Go back to your Chapter 2 file (`02-react-agent.ts`) and update the `AgentExecutor` construction to:

```
const agentExecutor = new AgentExecutor({
 agent,
 tools,
 verbose: true,
 handleParsingErrors: true,
});
```

Re-run it. With smaller or less capable models, you'll occasionally see a recovered parsing error in the verbose trace (the framework noticing malformed output and prompting a retry) instead of an outright crash. With `gpt-4o-mini`, formatting mistakes are rare, so don't be surprised if you don't actually witness one — the setting is cheap insurance regardless.

#### Common mistakes and fixes:

- *Setting `maxIterations` too low for a genuinely multi-step task* — the agent will get cut off mid-task and return an incomplete answer. Balance the cap against the realistic complexity of your real-world tasks.
- *Confusing `maxIterations` with `maxExecutionTime`* — they're independent limits; either one alone can stop the loop, whichever is hit first.
- *Forgetting `returnIntermediateSteps: true` before trying to read `result.intermediateSteps`* — without it, that field will be empty or undefined.

## 6.3 Knowledge Check

**Summary:** Agent loops need explicit safety limits because an LLM can get stuck retrying a confusing tool indefinitely. `maxIterations` caps the number of reasoning cycles, `maxExecutionTime` caps wall-clock time, and `handleParsingErrors` lets classic ReAct agents recover gracefully from malformed text output instead of crashing. `returnIntermediateSteps` exposes the full tool-call history for debugging.

#### Key takeaways:

- Every production agent should have a `maxIterations` (and usually a `maxExecutionTime`) set — never leave these at "infinite" by accident.
- `handleParsingErrors` matters most for classic ReAct agents; native tool-calling agents are far less prone to this specific failure.
- Hitting a stopping condition isn't a "crash" — it's `AgentExecutor` gracefully returning its best partial result.

- `returnIntermediateSteps` is your primary debugging tool for understanding *why* an agent behaved a certain way, beyond what `verbose` shows live.

### Practice questions:

1. What's the difference in what `maxIterations` and `maxExecutionTime` each protect against?
2. Why would `handleParsingErrors` matter less for a `createToolCallingAgent`-based agent?
3. If an agent hits `maxIterations` without producing a clean final answer, what should your application logic do with that result?

**Mini exercise:** Change `flakyLookupTool`'s description to actually explain the expected ID format (e.g., "IDs must be in the format XXX-000") and re-run the lab. Observe whether a clearer description changes how many tool calls the model makes before giving up or succeeding.

---

## Chapter 7: Multi-Tool Agents — Combining Skills and Tracing Reasoning

### 7.1 Concept and Theory

**The analogy.** A single-tool assistant is like a person with only a calculator on their desk — useful, but narrow. A multi-tool assistant has a calculator, a phone, and a filing cabinet all within reach, and — crucially — the judgment to know which one applies to which part of a request. The hard part isn't giving someone access to many tools; it's making sure they reliably pick the *right* one for each sub-task, in the right order.

**How an agent actually decides which tool to use, among many.** There's no separate "routing model" or special selection logic — it's the exact same mechanism from Chapter 3: the model reads every tool's `name` and `description` (all of them, every turn) and decides which one (if any) best matches what it currently needs. This is why precise, non-overlapping descriptions matter even more once you have several tools — if two tools have similar-sounding descriptions, the model may pick the wrong one, or alternate between them indecisively.

**Multi-step, multi-tool reasoning.** A genuinely useful agent task often requires *chaining* tools: search for a fact, then calculate something using that fact, then format the result. Each of these is a separate Thought → Action → Observation (or tool-call) cycle, and the model carries forward everything learned in previous cycles via the `agent_scratchpad`. This is the payoff of everything you've learned so far — Chapters 3 through 6 individually taught you how to build, call, and safely bound *one* tool's use; this chapter is about letting several of them work together on one request.

**Why tracing matters even more here.** With one tool, it's usually obvious what happened. With three or more tools, "why did it call X before Y?" or "why didn't it use Z at all?" becomes a real question you need to answer when debugging or improving an agent. `verbose: true` and `returnIntermediateSteps: true` (from Chapter 6) are how you answer that question

concretely, rather than guessing.

## 7.2 Practical Lab: A Three-Tool Research & Calculation Agent

### Step 1 — Create `07-multi-tool-agent.ts`

```
import "dotenv/config";
import { z } from "zod";
import { tool } from "@langchain/core/tools";
import { ChatOpenAI } from "@langchain/openai";
import { TavilySearch } from "@langchain/tavily";
import { ChatPromptTemplate, MessagesPlaceholder } from
"@langchain/core/prompts";
import { createToolCallingAgent, AgentExecutor } from "langchain/agents";

const calculatorTool = tool(
 async ({ operation, a, b }) => {
 switch (operation) {
 case "add":
 return String(a + b);
 case "subtract":
 return String(a - b);
 case "multiply":
 return String(a * b);
 case "divide":
 if (b === 0) return "Error: cannot divide by zero.";
 return String(a / b);
 default:
 return `Error: unknown operation "${operation}".`;
 }
 },
 {
 name: "calculator",
 description:
 "Performs one arithmetic operation (add, subtract, multiply, or divide) on two numbers. Use this for any math – never compute math yourself without it.",
 schema: z.object({
 operation: z.enum(["add", "subtract", "multiply", "divide"]),
 a: z.number(),
 b: z.number(),
 }),
 }
);

const textStatsTool = tool(
 async ({ text }) => {
 const wordCount = text.trim().split(/\s+/).filter(Boolean).length;
 return JSON.stringify({ wordCount });
 },
 {
 name: "text_stats",
 description: "Counts the number of words in a given block of text.",
 schema: z.object({ text: z.string() }),
 }
);

const searchTool = new TavilySearch({ maxResults: 3 });

async function main() {
 const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });

 const prompt = ChatPromptTemplate.fromMessages([
```

```

 "system",
 "You are a careful research assistant. Use search for facts you're unsure
of, calculator for any math (never compute math yourself), and text_stats to
count words. Explain which tools you used and why, briefly, before your final
answer.",
],
 ["human", "{input}"],
 new MessagesPlaceholder("agent_scratchpad"),
]);

const tools = [calculatorTool, textStatsTool, searchTool];

const agent = createToolCallingAgent({ llm, tools, prompt });

const agentExecutor = new AgentExecutor({
 agent,
 tools,
 verbose: true,
 maxIterations: 8,
 returnIntermediateSteps: true,
});

const result = await agentExecutor.invoke({
 input:
 "Search for the current population of Canada, divide that number by 1000,
and also tell me how many words are in this sentence: 'Agents are a powerful
LangChain concept.'",
});

console.log("\nFINAL ANSWER:\n", result.output);

console.log("\n--- TOOL CALL TRACE ---");
for (const step of result.intermediateSteps) {
 console.log(`Tool: ${step.action.tool}`);
 console.log(`Input: ${JSON.stringify(step.action.toolInput)}`);
 console.log(`Result: ${step.observation}`);
 console.log("---");
}
}
}

main();

```

**Why the system message explicitly names all three tools and when to use each:** with multiple tools available, leaving tool selection entirely to inference from descriptions alone works, but adding a short orienting sentence in the system prompt (as we did) measurably improves reliability — especially the instruction "never compute math yourself," which discourages the model from quietly reverting to Chapter 1's unreliable mental arithmetic.

**Why we manually loop over `result.intermediateSteps` and print each one:** this turns the raw trace data structure into a clean, readable audit log — exactly the kind of thing you'd log in a real production system to understand agent behavior after the fact, separate from the live verbose console noise.

## Step 2 — Run it

```
npx ts-node 07-multi-tool-agent.ts
```

**Expected result:** the trace at the bottom should show (in some order) a call to `tavily_search` (or similar) for Canada's population, a call to `calculator` with `operation: "divide"`



using that population figure, and a call to `text_stats` on the given sentence (which should report 6 words). The final answer should weave all three results into one coherent reply.

**How to verify it worked correctly:** check that - - - `TOOL CALL TRACE` - - - lists exactly the three tools you'd expect, with sensible inputs and outputs for each — if `text_stats` reports a word count other than 6, or `calculator` wasn't called for the division, that's a sign your prompt or descriptions need sharpening, not that something is "broken."

#### Common mistakes and fixes:

- *The model tries to do the division itself instead of calling `calculator`* — strengthen the system prompt's instruction even further (e.g., "You MUST use the calculator tool for any arithmetic, with no exceptions") and re-run; system prompt wording genuinely changes tool-selection behavior.
- *`step.action.tool` is undefined* — double check you're iterating `result.intermediateSteps`, which only exists because `returnIntermediateSteps: true` was set; without that flag this array will be missing entirely.
- *Search returns a slightly different population figure than you expected* — this is normal; population figures vary by source and date. Focus on whether the *pipeline* (search → calculate) worked, not on matching one "true" number exactly.

## 7.3 Knowledge Check

**Summary:** Tool selection among many tools works exactly like single-tool selection — the model reads every available tool's name and description each turn and picks accordingly — so precise, non-overlapping descriptions matter even more as your toolset grows. Multi-step tasks chain several Thought/Action/Observation (or tool-call) cycles together, carried forward via the scratchpad. `returnIntermediateSteps` turns this into an inspectable, loggable trace.

#### Key takeaways:

- More tools means descriptions need to be more precise and clearly differentiated from each other, not less.
- A short, explicit system prompt about *when* to use each tool meaningfully improves reliability beyond descriptions alone.
- The agent doesn't "plan ahead" in one shot — it decides one tool call at a time, reacting to each new observation.
- Logging `intermediateSteps` is the production-grade equivalent of the `verbose` trace you've relied on throughout this guide.

#### Practice questions:

1. Why might overlapping tool descriptions cause unreliable tool selection?
2. What's the practical difference between `verbose: true` and `returnIntermediateSteps: true` — when would you want one, the other, or both?

3. In the lab's final task, why did the agent need to call `calculator` *after* search, and not the other way around?

**Mini exercise:** Add your `wordReverser` tool from Chapter 3's exercise into this three-tool agent, and ask a single question that requires all four tools in one request. Inspect the trace to confirm the order each tool was called in, and whether that order makes logical sense for the task.

---

## Chapter 8: Final Project — Build a "Research & Calculate" Assistant

You've now covered every core concept in this phase: agents vs. chains, the ReAct loop, custom tools, native tool calling, real-world search tools, safety limits, and multi-tool orchestration. This project combines all of it into one polished, reusable script.

### Project Brief

Build a command-line research assistant that can:

1. Answer general questions using its own knowledge when appropriate.
2. Search the web for anything time-sensitive or uncertain.
3. Perform any arithmetic accurately using a calculator tool — never by "mental math."
4. Convert between units (build a new `unitConverter` tool — e.g., kilometers ↔ miles, kilograms ↔ pounds) as practice defining one more original tool from scratch.
5. Be protected by sensible `maxIterations` and `maxExecutionTime` limits.
6. Print a clean, human-readable trace of every tool call it made, after giving its final answer.

### Step-by-Step Build Guide

#### Step 1 — Scaffold the file

Create `08-final-project.ts` in the same project. Reuse `calculatorTool` and `searchTool` exactly as built in Chapter 7 — copy them in directly.

#### Step 2 — Build the new `unitConverter` tool

This is your own design exercise, following the Chapter 3 pattern exactly: pick a clear `name`, write a precise `description` listing exactly which conversions it supports, define a `schema` with an enum of supported conversion types and a numeric value, and write the conversion math in the function body. Test it directly with `.invoke()` before moving on, the same way you tested every other tool in this guide.

#### Step 3 — Write a strong system prompt

Following the lesson from Chapter 7, explicitly tell the model: use search for anything current or uncertain; use calculator for all arithmetic, no exceptions; use `unit_converter` for any measurement conversion; and to briefly note which tools it used before giving the final answer.

#### Step 4 — Assemble the agent

Use `createToolCallingAgent + AgentExecutor`, exactly as in Chapters 4, 5, and 7 — by now this part should feel routine. Set `maxIterations: 8`, a reasonable `maxExecutionTime`, and `returnIntermediateSteps: true`.

### Step 5 — Test it with a genuinely multi-tool question

For example: *"What is the current distance record for the long jump, in meters? Convert that to feet, and then tell me what that distance times 10 would be."* This single question should require search (for the record), unit conversion (meters → feet), and calculation ( $\times 10$ ) — all three of your tools, in a sensible order.

### Step 6 — Print the trace

Reuse the exact `intermediateSteps` loop pattern from Chapter 7 to print a clean audit log after the final answer.

## Verifying Your Final Project

- The final answer should be coherent and reference real, specific information (proving search worked).
- The unit conversion in the answer should be mathematically correct (you can verify by hand:  $\text{meters} \times 3.28084 \approx \text{feet}$ ).
- The trace should show three distinct tools being called, each with sensible inputs.
- If you intentionally ask a simple question with no time-sensitive or numeric component (e.g., "what is LangChain?"), the agent should answer confidently *without* needlessly calling any tools — confirming your prompt strikes the right balance.

## Stretch Goals (Optional, Beyond This Phase)

- Add conversation memory (you already know this from your Core phase) so the assistant remembers earlier questions in the same session.
- Swap `createToolCallingAgent + AgentExecutor` for LangGraph's prebuilt `createReactAgent` (a different, more modern agent runtime you'll meet if you continue into a LangGraph-focused roadmap) and compare the developer experience.
- Add a fourth, riskier tool (e.g., one that "sends an email") and practice the human-approval pattern: pause before executing it and ask for console confirmation.

---

## Appendix A: Troubleshooting Cheat Sheet

Symptom	Likely Cause	Fix
Missing input variables error on agent creation	Your prompt is missing <code>agent_scratchpad</code> (and/or <code>input</code> )	Add new <code>MessagesPlaceholder("agent_scratchpad")</code> and <code>["human", "{input}"]</code> to your prompt

Symptom	Likely Cause	Fix
Agent never calls any tool	Description too vague, or system prompt doesn't hint at tool use	Sharpen tool descriptions and add explicit "use X for Y" guidance in the system prompt
Agent calls the same tool repeatedly without progress	Tool's error message doesn't explain how to correct the input	Make tool error messages instructive ("expected format: XXX-000")
Process seems to hang indefinitely	No <code>maxIterations</code> / <code>maxExecutionTime</code> set	Always set both in production code
<code>TypeError</code> inside your tool function	Zod schema and function parameter names mismatched	Ensure destructured parameter names exactly match your <code>z.object({...})</code> keys
Parsing errors crash a classic ReAct agent	<code>handleParsingErrors</code> not set	Add <code>handleParsingErrors: true</code> to <code>AgentExecutor</code>
<code>result.intermediateSteps</code> is empty/undefined	<code>returnIntermediateSteps</code> not set	Add <code>returnIntermediateSteps: true</code> to <code>AgentExecutor</code>

## Appendix B: Glossary

- **Agent** — a loop, built around an LLM, that dynamically decides whether and which tool to call before producing a final answer.
- **AgentExecutor** — the LangChain.js component that runs the agent's reasoning loop repeatedly until a final answer or a stopping condition is reached.
- **ReAct** — a prompting pattern alternating Reasoning ("Thought") and Acting ("Action"), with results fed back in ("Observation").
- **Tool** — a named, described, schema-validated function the agent may choose to call.
- **Tool calling (native)** — a model-provider feature allowing structured tool-call requests/responses instead of parsed free text.
- **Agent scratchpad** — the running history of tool calls and results within the current reasoning loop, fed back into the model each turn.
- **Stopping condition** — any limit (`maxIterations`, `maxExecutionTime`) that forces the agent loop to end even without a clean final answer.

## Appendix C: Further Reading

- LangChain.js agents reference documentation (search "LangChain.js AgentExecutor")
- LangChain Hub, for browsing other community prompts beyond `hwchase17/react`
- Your own roadmap's next phase: Production (Callbacks & Observability, Streaming &

Batching) — these pair naturally with everything you just learned about tracing agent behavior